

UNIVERSITY OF PLYMOUTH

BSC (HONS) COMPUTER SCIENCE (SOFTWARE ENGINEERING)
MAL3018 COMPUTING PROJECT

Name : Teoh Yi Shan

Student No : BSSE2506022

Project Name: SIRAG: A Closed-Domain Retrieval-Augmented Generation System Using an On-Device Large Language Model for Student Information Services

---

Name of Supervisor : Ts. Mohd Khairi Bin Kamaruddin

Date : 28th October 2026

Table of Contents

1. Project Title 3
2. Keywords 3
3. Link 3
4. Student Information 3
5. Supervisor Information 3
6, Problem Statement 3
7. Objectives 4
8. Scope of Project 5
9. Literature Review 6
10. Methodology 8
11. Expected Deliverables 12
13. Proposed Gantt Chart 14
14. Resource Required 15
15. Expected Outcome and Significance 16
16. Reference 17

1. Project Title

SIRAG: A Closed-Domain Retrieval-Augmented Generation System Using an On-Device Large Language Model for Student Information Services

2. Keywords: Retrieval-Augmented Generation (RAG), Closed-Domain RAG, On-Device Large Language Model (LLM), Edge Computing, Privacy-Preserving AI, Local LLM / Offline LLM, Vector Database, Raspberry Pi Kiosk, Speech-to-Text and Text-to-Speech, Hallucination Mitigation, FastAPI / LangChain / Ollama

3. Link:

PebblePad Link: https://v3.pebblepad.co.uk/spa/#/public/mtw79Zrj5jG5hf3yHMRkhwW9tc

GitHub Link: https://github.com/GitCode330/MAL3018-Computing-Project.git

4. Student Information

- Name: Teoh Yi Shan
- Student ID: BSSE2506022
- Programme / Major: Software Engineering
- Year / Semester: Y3S1
- Email: BSSE2506022@peninsulamalaysia.edu.my

5. Supervisor Information

- Name**:** Mohd Khairi bin Kamaruddin
- Department: School of Technology and Engineering (SOTE)
- Email: khairi@peninsulamalaysia.edu.my

6, Problem Statement

In a university environment, student information services are constantly overwhelmed by a high volume of repetitive, low-complexity inquiries. Students frequently ask basic questions regarding campus locations, timetable schedules, and administrative procedures. Furthermore, students often lack immediate access to clear information regarding college regulations, academic rules, and institutional policies, which are typically buried in lengthy, static PDF handbooks or scattered across different web pages. This results in students experiencing frustration and delayed responses, while administrative staff are diverted from high-value tasks to answer the same questions repeatedly.

While digital solutions exist, current methods are fundamentally inadequate. Traditional keyword-based search engines fail to understand natural language queries for example like student asking, "Where can I get lunch?" instead of searching for "cafeteria location". Conversely, utilizing public Cloud-based Large Language Models (LLMs) like ChatGPT introduces severe risks for educational institutions:

- Data Privacy and Security: Transmitting internal student queries to external cloud servers risks exposing institutional data and violates data sovereignty.
- Hallucination of Policies: General-purpose cloud models lack specific, up-to-date knowledge of a closed campus domain. They often "hallucinate" or invent non-existent rules and policies, which is highly dangerous for academic regulations.
- Cost and Reliability: Reliance on subscription-based APIs creates an ongoing financial burden that scales with usage, and cloud-dependent systems are vulnerable to internet outages.

Therefore, there is a critical need for a closed-domain, privacy-preserving student information system. This project proposes SIRAG (Student Information Retrieval-Augmented Generation), a locally hosted system that combines a dynamic vector database of official college documents with an on-device Large Language Model. This ensures that students receive instant, natural-language, and factually accurate answers regarding rules and policies, strictly grounded in official documentation, without relying on external cloud infrastructure.

7. Objectives

- Develop a locally hosted Large Language Model (LLM) inference engine on an edge computing platform for processing and generating responses to student enquiries.
- Develop a customised institutional knowledge base integrated with a Retrieval-Augmented Generation (RAG) mechanism for retrieving relevant information and generating accurate, closed-domain responses to student enquiries.
- Implement an interactive voice-based user interface integrating speech-to-text and text-to-speech technologies for natural two-way communication between students and the AI system.
- Integrate the local LLM, RAG mechanism, and voice interaction modules into a unified student information system for real-time user enquiries.
- Evaluate the developed system in terms of response accuracy, retrieval relevance, response time, and usability in providing student information services.

8. Scope of Project

In-Scope:

- Development of a Retrieval-Augmented Generation pipeline. This includes document ingestion, embedding generation, vector database storage, semantic retrieval, and LLM answer generation.
- Collection and formatting of college-specific data into a vector database.
- Selection and local deployment of a lightweight, open-source LLM using efficient inference engines running on local hardware.
- Testing the system for Retrieval Accuracy (does it find the right document?).

Out-Of-Scope:

- This project utilizes pre-trained open-source models and relies entirely on the RAG architecture for domain adaptation, rather than computationally expensive fine-tuning or training from scratch.
- The system will strictly operate offline with no reliance on OpenAI, Google Gemini, or other external cloud AI providers.
- The system will not process images, videos, or complex diagrams; it will only handle text-based documents and textual queries.
- The system is strictly restricted to a closed domain. It will only answer college-related inquiries based strictly on the retrieved context from the provided RAG knowledge base. It is not designed to act as a general-purpose AI and will explicitly refuse to answer general knowledge or out-of-scope questions.

9. Literature Review

The development of an effective student information service requires a careful examination of current approaches, their limitations, and the technological shifts toward localized Artificial Intelligence. This review synthesizes existing literature across four key domains: traditional service desks, cloud-based Language Models (LLMs), on-device AI, and edge computing hardware.

1. Traditional Student Information Service Desks

Historically, university helpdesks rely on direct human interaction. Research indicates that human staff are highly effective at providing empathetic, personalized support and handling complex, non-standard inquiries that require nuanced judgment (Aisyah, 2026). However, this traditional model suffers from severe operational bottlenecks. Studies show that a vast majority of helpdesk queries are repetitive and low-complexity example like, asking for office locations or timetable schedules, which consumes significant staff time and leads to delayed response times during peak periods. Furthermore, the reliance on human memory results in inconsistent information delivery, as different staff members may provide varying answers to the same policy questions.

2. Cloud-Based Large Language Models (LLMs)

Institutions have explored cloud-based LLMs to automate these services example like ChatGPT, Google Gemini. These models offer state-of-the-art natural language understanding and generation, requiring no local hardware investment, and benefit from continuous improvements by the provider. However, deploying them in an educational setting introduces critical weaknesses. The primary concern is data privacy and sovereignty, as transmitting internal queries to third-party servers risks exposing institutional data. Additionally, general-purpose cloud models lack specific campus knowledge, leading to "hallucinations" that means confidently generating false information. Finally, they impose ongoing operational costs and are vulnerable to internet outages and service disruptions (Local vs. Cloud-Based LLMs).

3. On-Device Small Language Models

Recent literature has focused on On-Device SLMs to mitigate privacy and cost issues. By running the model locally, SLMs ensure strict privacy preservation and low latency, as data does not leave the local network. They offer energy efficiency, offline operation, and competitive accuracy for narrow-domain tasks. However, SLMs are constrained by the hardware they run on. They face limited memory and compute, forcing capability trade-offs. Consequently, running larger models on edge devices leads to significant inference latency, degrading the user experience (Johnson, 2025).

4. Raspberry Pi as Edge Terminal for AI Kiosks

The Raspberry Pi has emerged as a popular, cost-effective edge terminal for AI kiosks. Its strengths lie in low-cost deployment and real-world feasibility, with newer hardware acceleration options such as the Raspberry Pi AI HAT+ enabling local neural network processing. Nevertheless, the Pi has computational limitations. It is fundamentally best suited as a client terminal rather than a primary inference server. Attempting to run LLMs directly on the Pi results in limited on-device model quality and slow processing speeds, necessitating a client-server architecture where the Pi handles the user interface and offloads heavy AI processing elsewhere (AI software - raspberry pi documentation).

To clearly distinguish the proposed SIRAG system from existing approaches, Table 1 summarises the strengths and weaknesses of the discussed technologies.

Approach	Key Strengths	Key Weakness	Relevance to SIRAG
Traditional Desks	Empathetic, handles complex queries.	Slow, inconsistent, operational bottleneck.	Demonstrates the need for automated, student support.
Cloud LLMs	Powerful NLP, no local hardware needed.	Privacy risks, hallucinations, API costs.	Highlights why SIRAG must be local and grounded in college data.
On-Device SLMs	Privacy, offline capability, zero API cost.	Hardware constraints, lower accuracy.	Validates the use of local, lightweight models
Raspberry Pi Kiosk	Cost-effective, excellent client interface.	Cannot run heavy LLMs natively.	Validates the client-server architecture (Pi as UI, server as host).
Proposed SIRAG System	Local privacy, RAG grounding (no hallucination), edge kiosk UI, offline.	Requires manual knowledge base curation.	**The Research Gap:** Combines local LLM + RAG + Edge Computing to solve the limitations of all the above.

(Table 1: Comparative Analysis of Existing Solutions vs. Proposed SIRAG System)

10. Methodology

Project Approach / Framework

The project will utilize the Agile Development Framework (Scrum), structured around two-week sprints to align with the MAL3018 module requirements. This iterative approach allows for continuous testing and refinement of the AI models. Each sprint will focus on specific milestones, such as data ingestion, RAG pipeline construction, edge-device UI development, and voice integration, ensuring that a functional Minimum Viable Product (MVP) is achieved early before adding complex features.

Tools and Technologies

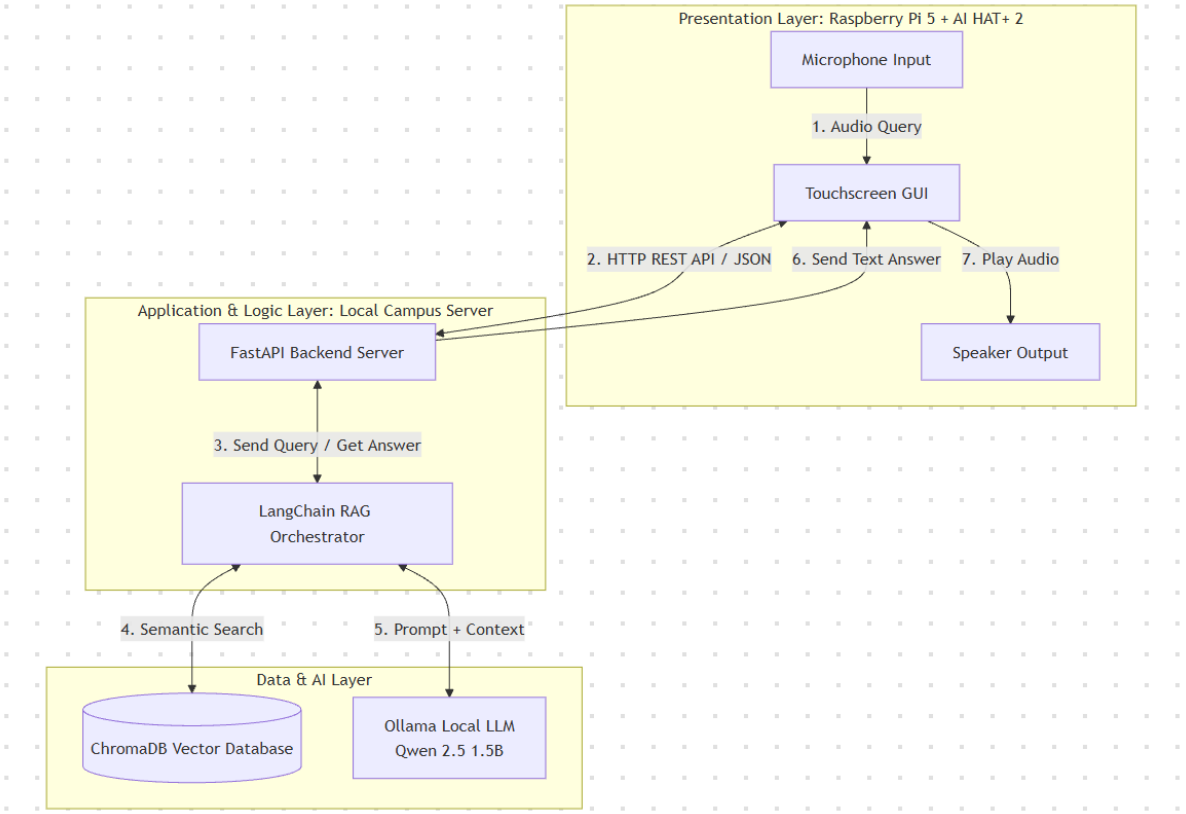
- **Edge Client (Raspberry Pi):** Python 3, Tkinter or Streamlit (for the touchscreen Graphical User Interface), Raspberry Pi OS (Linux Debian).
- **Backend Server:** FastAPI (for creating fast, asynchronous RESTful APIs to handle requests from the Pi).
- **AI & RAG Framework:** LangChain or LlamaIndex (Python) for orchestrating the retrieval pipeline.
- **Local LLM Engine:** Ollama (to run lightweight open-source models like Qwen 2.5 or Llama 3.2 locally without internet).
- **Vector Database:** ChromaDB or FAISS (for storing and retrieving document embeddings).
- **Speech Processing:** OpenAI Whisper (Speech-to-Text), Piper TTS (Text-to-Speech) running locally on the Pi/server.
- **Version Control & Testing:** Git & GitHub (version control), PyTest (unit testing), Postman (API testing).

System Design / Architecture

The system follows a distributed **Three-Tier Architecture**, optimized to overcome the hardware limitations of edge devices:

1. **Presentation Layer (Edge Client):** A Raspberry Pi 5 with a touchscreen acts as the user-facing kiosk. It handles the graphical user interface (GUI), microphone input, and speaker output.
2. **Application & Logic Layer (Local Server):** A FastAPI backend runs on a local PC/server. It receives text/audio queries from the Pi, converts audio to text (if necessary), and manages the RAG orchestration.

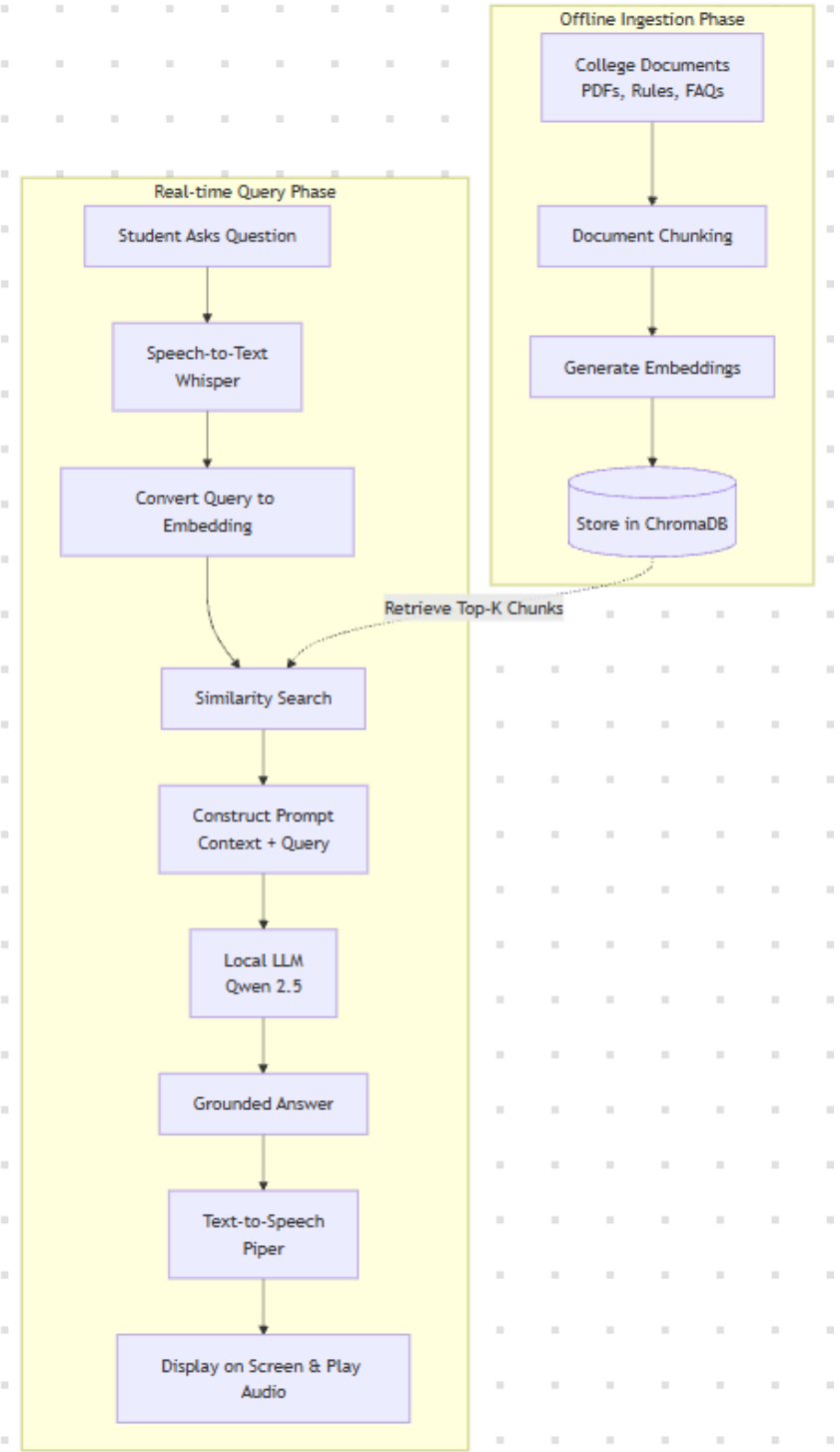
3. **Data & AI Layer (Local Server):** The RAG pipeline queries the ChromaDB vector database for relevant campus documents. The retrieved context and the user's query are then passed to the local LLM (via Ollama) to generate a grounded response, which is sent back to the Pi.



(Figure 1: System Architecture Diagram)

SIRAG Process Flow

The operational flow of the system is divided into two distinct phases: the Document Ingestion Phase (offline) and the Query Processing Phase (real-time).



(Figure 2: SIRAG Process Flow Diagram)

- Phase 1: Ingestion (Offline): College documents (PDFs, rules, FAQs) are loaded, split into smaller text chunks, converted into vector embeddings, and stored in the ChromaDB vector database.
- Phase 2: Querying (Real-time): When a student asks a question via the Pi, the query is embedded. The system performs a similarity search in ChromaDB to retrieve the most relevant text chunks. These chunks are combined with the original query into a prompt and sent to the local LLM. The LLM generates an answer strictly based on the retrieved text and sends it back to the kiosk.

Implementation Plan

- **Sprint 1 (Foundation):** Set up GitHub repository, configure the local server, and curate the college FAQ dataset (PDFs, text files).
- **Sprint 2 (RAG Pipeline):** Implement document chunking, embedding generation, and set up the Vector Database. Develop the retrieval logic.
- **Sprint 3 (Local LLM & API):** Deploy the open-source LLM via Ollama. Develop the FastAPI to connect the RAG pipeline and LLM, enabling text-based querying.
- **Sprint 4 (Edge Client & Voice):** Configure the Raspberry Pi UI. Integrate local Speech-to-Text (Whisper) and Text-to-Speech (Piper) modules for hands-free interaction.
- **Sprint 5 (Integration & Optimization):** Connect the Raspberry Pi client to the FastAPI server. Optimize the RAG parameters (chunk size, top-k retrieval) to minimize latency and hallucinations.

Testing and Evaluation

- **Unit Testing:** Validate individual Python functions, such as document chunking, embedding generation, and API endpoints using PyTest.
- **Integration Testing:** Ensure seamless communication between the Raspberry Pi client, the FastAPI server, and the Ollama LLM engine.
- **Functional & Accuracy Testing:** Evaluate the RAG system against a controlled dataset of 50 student queries. Measure Retrieval Accuracy (did it find the right document?) and Groundedness (did the LLM hallucinate, or did it stick to the retrieved facts?).
- **Performance Evaluation:** Measure system response time (latency) and hardware resource consumption (CPU/RAM usage) on both the Raspberry Pi and the local server to ensure the system meets the usability threshold (e.g., responses under 3 seconds).

11. Expected Deliverables

At the end of this project, the following deliverables will be produced and submitted via the ePortfolio:

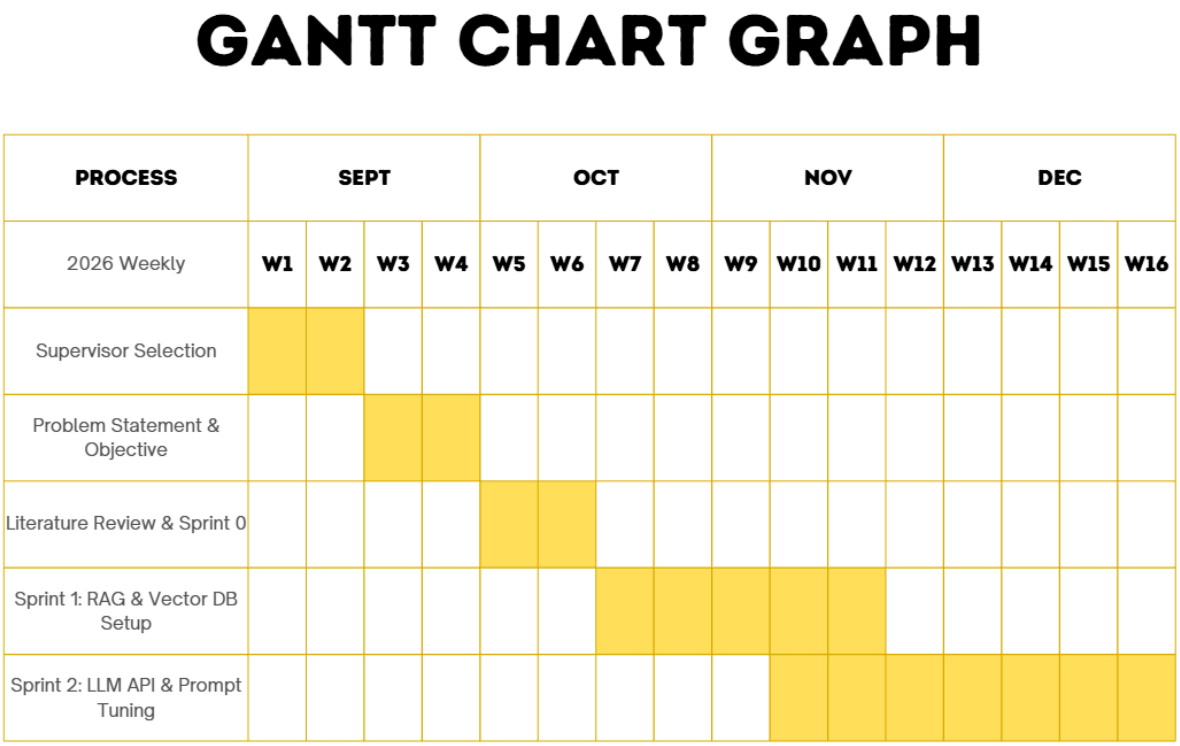
- Working Software Prototype: A fully functional SIRAG system running on a Raspberry Pi (client) communicating with a local server (LLM).
- Version-Controlled Repository: A GitHub repository containing all source code, an informative readme.md, and version histories of the RAG pipeline, API, and UI.
- Curated Knowledge Base: A structured dataset of college-specific FAQs and documents used for the vector database.
- Project Poster & Thumbnail: An A1 landscape poster summarizing the problem, solution, methodology, and results for the Showcase.
- Final Report: A comprehensive document detailing the entire software development lifecycle, literature review, results, and critical evaluation.
- ePortfolio Evidence: Completed Sprint Diary, CPD Reflections, and the Generative AI Declaration form.

12. Risk Plan

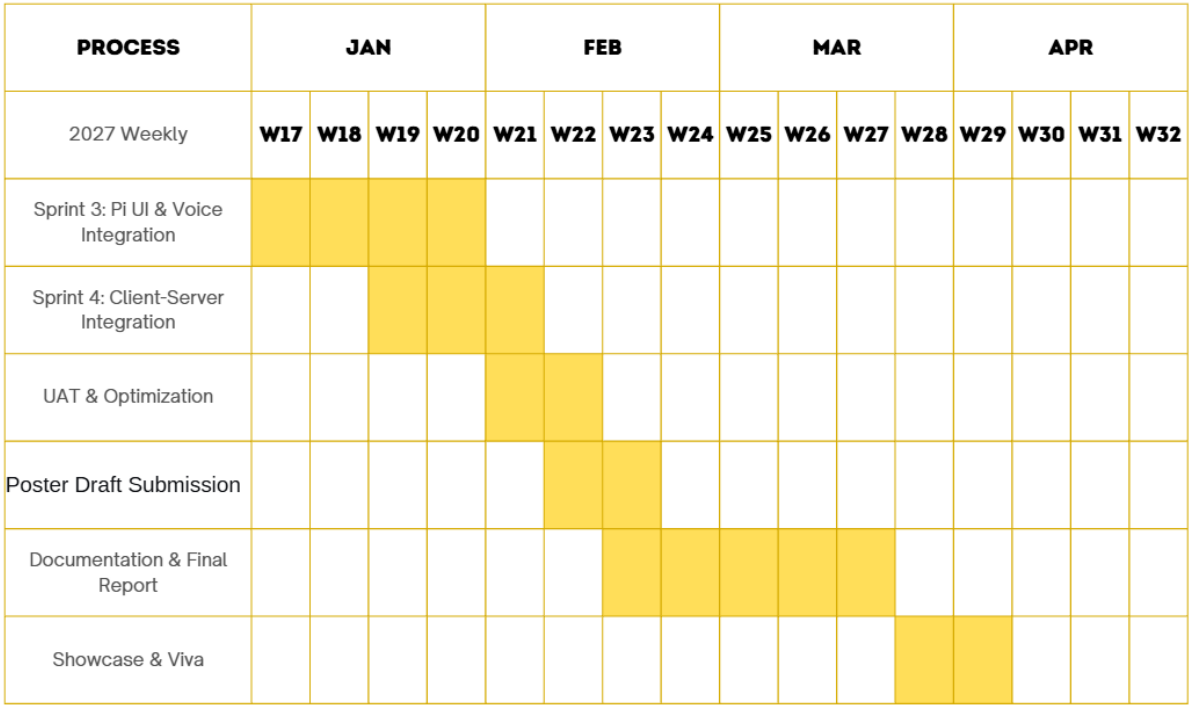
Risk ID	Description	Likelihood	Impact	Mitigation Strategy
R1	**Hardware Limitations:** The Raspberry Pi cannot process the LLM fast enough, causing severe latency.	High	High	**Mitigation:** Offload LLM inference to a local PC server. The Pi will only run the lightweight client UI and STT/TTS models.
R2	**Retrieval Failure:** The RAG system retrieves irrelevant documents, leading to hallucinated answers.	Medium	High	**Mitigation:** Carefully tune chunk sizes, test different embedding models, and implement a strict prompt template that instructs the LLM to say "I don't know" if the context is missing.
R3	**Data Quality:** Insufficient or outdated college FAQ data.	Medium	Medium	**Mitigation:** Start with public data (college website) and generate synthetic FAQs early in Sprint 1. Use the MVP approach.
R4	**Scope Creep:** Trying to add too many features (e.g., live database integration).	High	Medium	**Mitigation:** Strictly adhere to the defined Scope. Focus on text/voice FAQs only. Live database integration is out-of-scope.
R5	**Time Management:** Falling behind the 2-week sprint schedule.	Medium	High	**Mitigation:** Maintain a detailed Sprint Diary. Prioritize core features first. If a feature fails, document the attempt and pivot.

(Table 2: Risk Planning)

13. Proposed Gantt Chart



(Figure 3: 2026 Gantt Chart)



(Figure 4: 2027 Gantt Chart)

14. Resource Required

Hardware:

- **Raspberry Pi AI Hat +2 (8GB RAM)** with a compatible touchscreen display (acts as the client).
- **Local Development PC** (Windows/Linux) with a dedicated GPU to host the LLM and Vector DB.
- MicroSD card, power supply, and peripherals for the Pi.

Software & Libraries:

- **Operating System:** Raspberry Pi OS (Linux/Debian), Windows/Linux for the host server.
- **LLM Engine:** Ollama or vLLM (for running open-source models like Qwen 2.5 1.5B or Llama 3.2 3B).
- **RAG Framework:** LangChain or LlamaIndex (Python).
- **Vector Database:** ChromaDB or FAISS.
- **Backend API:** FastAPI or Flask.
- **Speech Processing:** OpenAI Whisper (STT), Piper TTS (local, lightweight).
- **Version Control:** Git & GitHub.

Data Resources:

- College official website content, student handbooks (PDFs), and synthesized FAQ datasets.

15. Expected Outcome and Significance

Expected Outcome:

The project is expected to produce a functional, locally hosted SIRAG system that successfully answers college-specific queries via a Raspberry Pi kiosk. The system will demonstrate high retrieval accuracy, minimal hallucination (by strictly grounding answers in the provided knowledge base), and acceptable latency (under 3-5 seconds per response) without relying on external internet connections.

Significance:

- **For the College:** The system provides a cost-effective, privacy-preserving student information service. It reduces the workload on administrative staff, allowing them to focus on complex inquiries.
- **For the Software Engineering Field:** This project serves as a practical implementation of Edge AI and RAG architecture. It demonstrates how to overcome hardware limitations by utilizing a client-server distributed architecture, contributing valuable insights into deploying generative AI in resource-constrained environments.
- **SDG Alignment:** The project directly supports **SDG 4 (Quality Education)** by improving access to information.

16. Reference

1. Aisyah, N. (2026) *What is a student information system? A complete guide for Malaysian Educational Institutions, HashMicro*. Available at: https://www.hashmicro.com/my/blog/sis-student-information-system/
2. *Local vs. Cloud-Based LLMs* (no date) *apxml*. Available at: https://apxml.com/zh/courses/getting-started-local-llms/chapter-1-introduction-large-language-models/local-vs-cloud-llms
3. Johnson, J. (2025) *Small language models (SLM): A comprehensive overview, Hugging Face – The AI community building the future.* Available at: https://huggingface.co/blog/jjokah/small-language-model
4. *AI software - raspberry pi documentation* (no date) *Rasberrypi*. Available at: https://www.raspberrypi.com/documentation/computers/ai.html
5. (2023) *Raspberry pi 5 heatsink and active cooler*. Available at: https://my.cytron.io/p-raspberry-pi-5-heatsink-and-active-cooler
6. *FASTAPI¶* (2026) *FastAPI*. Available at: https://fastapi.tiangolo.com/#recap
7. (2026) *What is Rag? A guide to retrieval augmented generation | datacamp*. Available at: https://www.datacamp.com/blog/what-is-retrieval-augmented-generation-rag
8. Staff, D. (2023) *What is retrieval augmented generation (rag)?*, *Databricks*. Available at: https://www.databricks.com/blog/what-is-retrieval-augmented-generation
9. *Raspberry Pi 5 AI HAT+2 Setup – Run Local AI & LLMs with 40 TOPS* (2026) *YouTube*. Available at: https://www.youtube.com/watch?v=820nG-bmWx4
10. Team, O. (2026) *What is a local Llm? how to run LLMS on your own hardware (2026)*, *OpenHands*. Available at: https://www.openhands.dev/blog/local-llm
11. Moez Al (2026) *Best vector databases 2026: Pinecone, Chroma, Qdrant & More | DataCamp*. Available at: https://www.datacamp.com/blog/the-top-5-vector-databases
12. OpenWhispr (2026) *How Whisper Ai Works: A complete guide*, *OpenWhispr*. Available at: https://openwhispr.com/blog/how-whisper-ai-works
13. *Piper text-to-speech* (no date) *Pipecat*. Available at: https://docs.pipecat.ai/api-reference/server/services/tts/piper